

MERN Stack Internship

DAY 9 ASSIGNMENT REPORT

Name: NIKHIL MANDLOI

Course: BTech CSIT 1 year

Position : MERNStackINTERN

Allocated Date:9/06/26

Submission Date: 9/006/26

MERN Stack Internship – Day Report

Task Title

Live Structural Redesign & State Tracking – React Kanban Board with Undo/Redo Functionality

Objective

The objective of this task was to enhance the previously developed React Kanban Board by implementing a localized state history tracking mechanism. The application was redesigned to maintain a stack-based history of task state changes, enabling real-time Undo and Redo operations without using any third-party libraries.

The task also focused on understanding React state immutability, reference management, and state transition tracking.

Technologies Used

- ☐ React.js
 - ☐ JavaScript (ES6)
 - ☐ CSS3
 - ☐ React Hooks (useState, useEffect)
 - ☐ Browser localStorage
-

Task Requirements

Existing Features

- ☐ Todo Column
- ☐ In Progress Column
- ☐ Completed Column
- ☐ Task Movement Between Columns
- ☐ Data Persistence Using localStorage

New Features Added

- ☐ Undo Functionality
 - ☐ Redo Functionality
 - ☐ State History Tracking
 - ☐ Localized Stack-Based State Management
 - ☐ Immutable State Updates
 - ☐ Reference-Safe Data Manipulation
-

Features Implemented

1. Kanban Board Structure

The board contains three workflow columns:

- ☐ Todo
- ☐ In Progress
- ☐ Completed

Each task is displayed as an individual card.

2. Task Movement System

Tasks can move between columns using action buttons.

Examples:

- ☐ Todo → In Progress
 - ☐ In Progress → Completed
 - ☐ In Progress → Todo
 - ☐ Completed → In Progress
-

3. Undo Functionality

An Undo button was implemented to restore the previous application state.

Whenever a task changes status:

- ☐ The previous state is preserved in a history stack.
- ☐ Clicking Undo restores the immediate previous state.

Example:

Todo → In Progress

Undo

In Progress → Todo

4. Redo Functionality

A Redo button was implemented to move forward through previously undone states.

Example:

Todo → In Progress

Undo

Todo

Redo

In Progress

5. History Stack Tracking

A dedicated history state was created to store snapshots of every task modification.

React State Variables:

- ☐ tasks
- ☐ history
- ☐ historyIndex

This structure allows navigation between previous and future states.

6. Immutable State Management

Task updates were implemented using:

- ☐ `map()`
- ☐ spread operator (...)

This ensures new array references are created instead of directly mutating existing state.

Benefits:

- ☐ Proper React re-rendering
 - ☐ Predictable state transitions
 - ☐ Safe memory reference handling
-

7. Local Storage Integration

The application continues to persist task data using `localStorage`.

Benefits:

- ☐ Tasks remain available after refresh.
 - ☐ User changes are preserved automatically.
-

Project Structure

`src/`

- |— `App.jsx`
- |— `App.css`
- |— `main.jsx`

Implementation Process

Step 1

Loaded existing task data from `localStorage`.

Step 2

Created history state management variables.

Step 3

Stored every task modification inside the history stack.

Step 4

Implemented Undo functionality using historyIndex decrement.

Step 5

Implemented Redo functionality using historyIndex increment.

Step 6

Updated UI with dedicated Undo and Redo control buttons.

Step 7

Applied enhanced styling and responsive layout improvements.

Step 8

Tested task transitions and state restoration workflows.

React Concepts Practiced

React Hooks

- ☐ useState()
- ☐ useEffect()

State Management

- ☐ Immutable Updates
- ☐ State History Tracking
- ☐ Snapshot Storage

Array Methods

- ☐ map()
- ☐ filter()
- ☐ slice()

Browser Storage

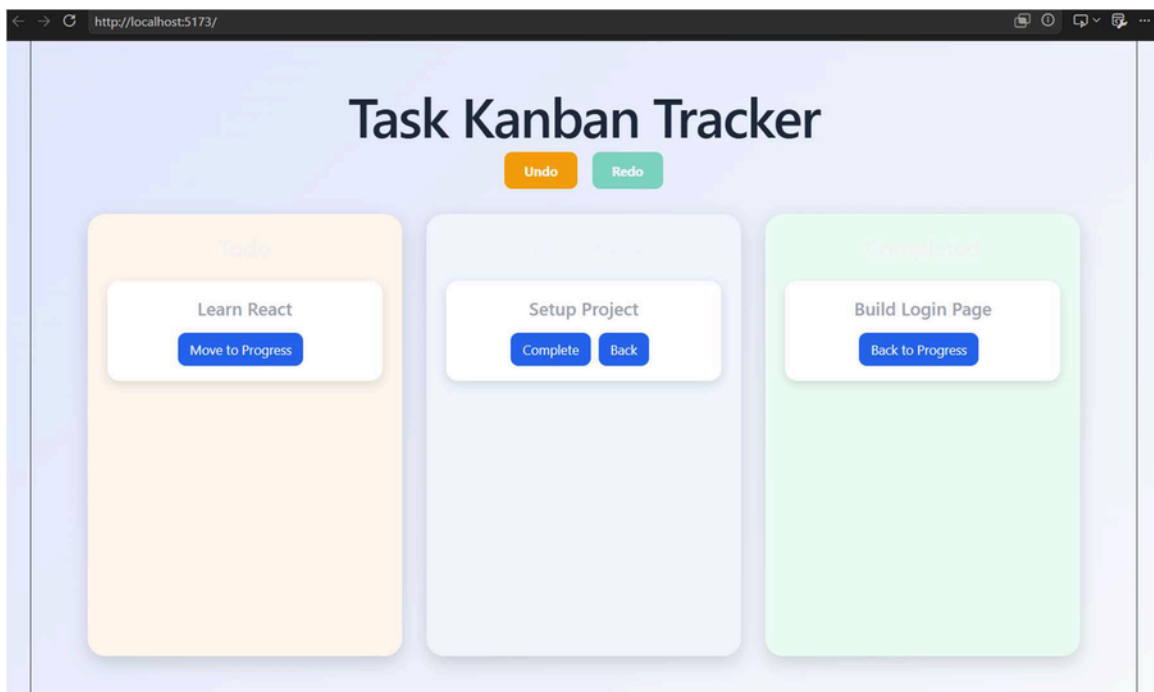
- ☐ localStorage.getItem()

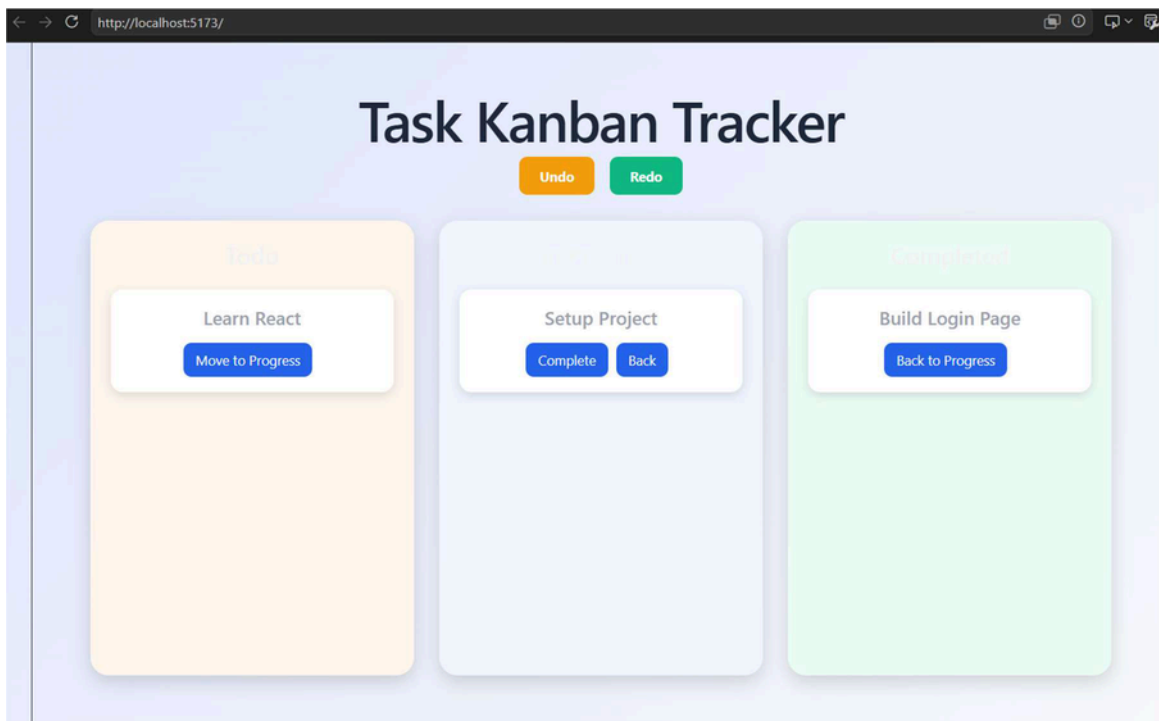
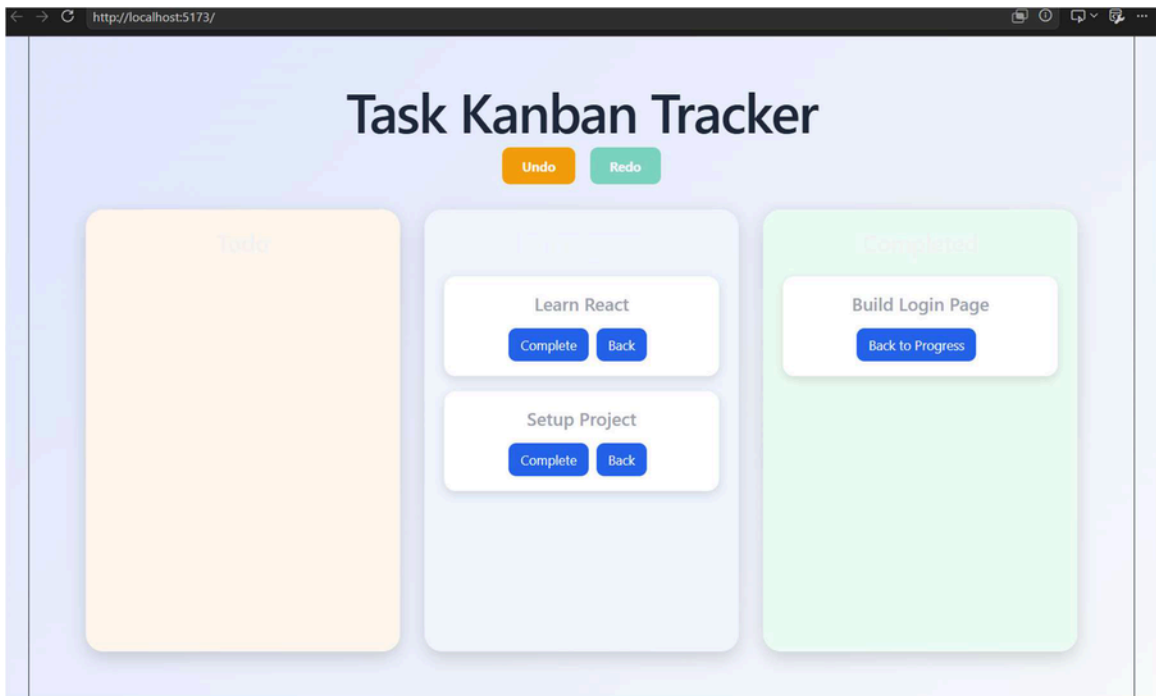
☐ `localStorage.setItem()`

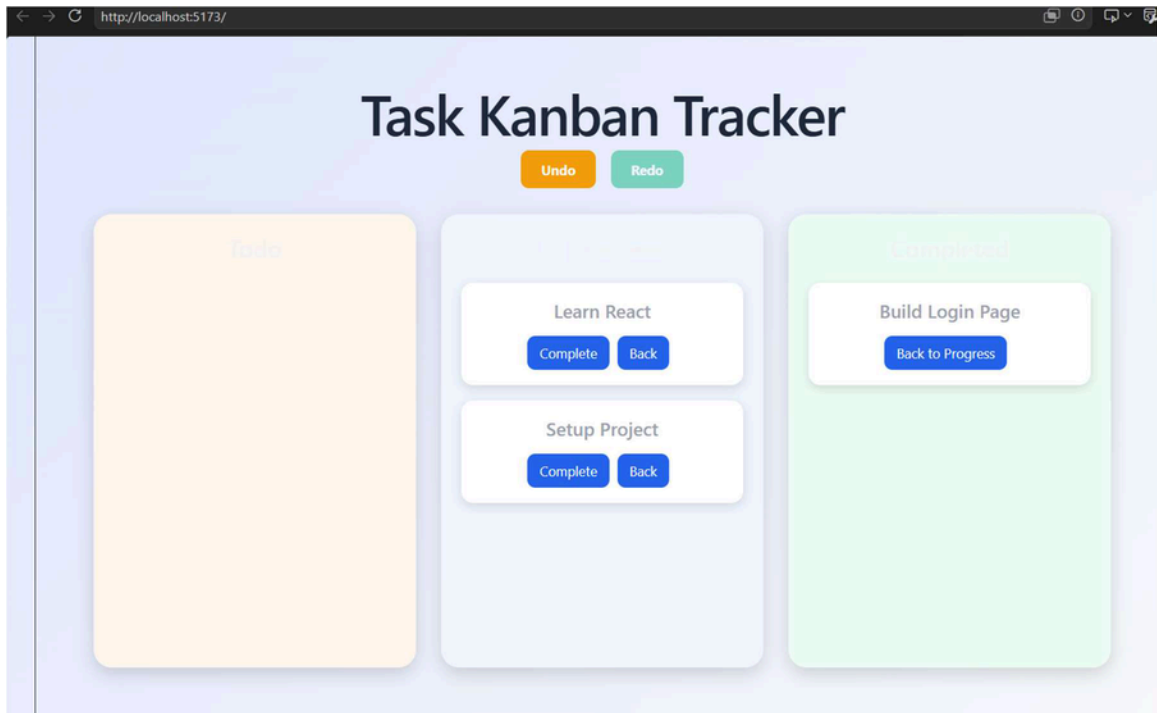
Learning Outcomes

Through this task I learned:

- ☐ Advanced React State Management
 - ☐ Undo and Redo Logic Implementation
 - ☐ Stack-Based History Tracking
 - ☐ Immutable State Updates
 - ☐ React Re-render Mechanism
 - ☐ Array Reference Management
 - ☐ Local Storage Persistence
 - ☐ Functional Programming Concepts
 - ☐ UI State Navigation Patterns
-







Conclusion

The React Kanban Board was successfully redesigned to support advanced state tracking through a localized history stack. Undo and Redo operations were implemented without external libraries by maintaining immutable snapshots of application state. The project demonstrates practical understanding of React state management, reference handling, stack-based navigation, and modern front-end architectural principles.